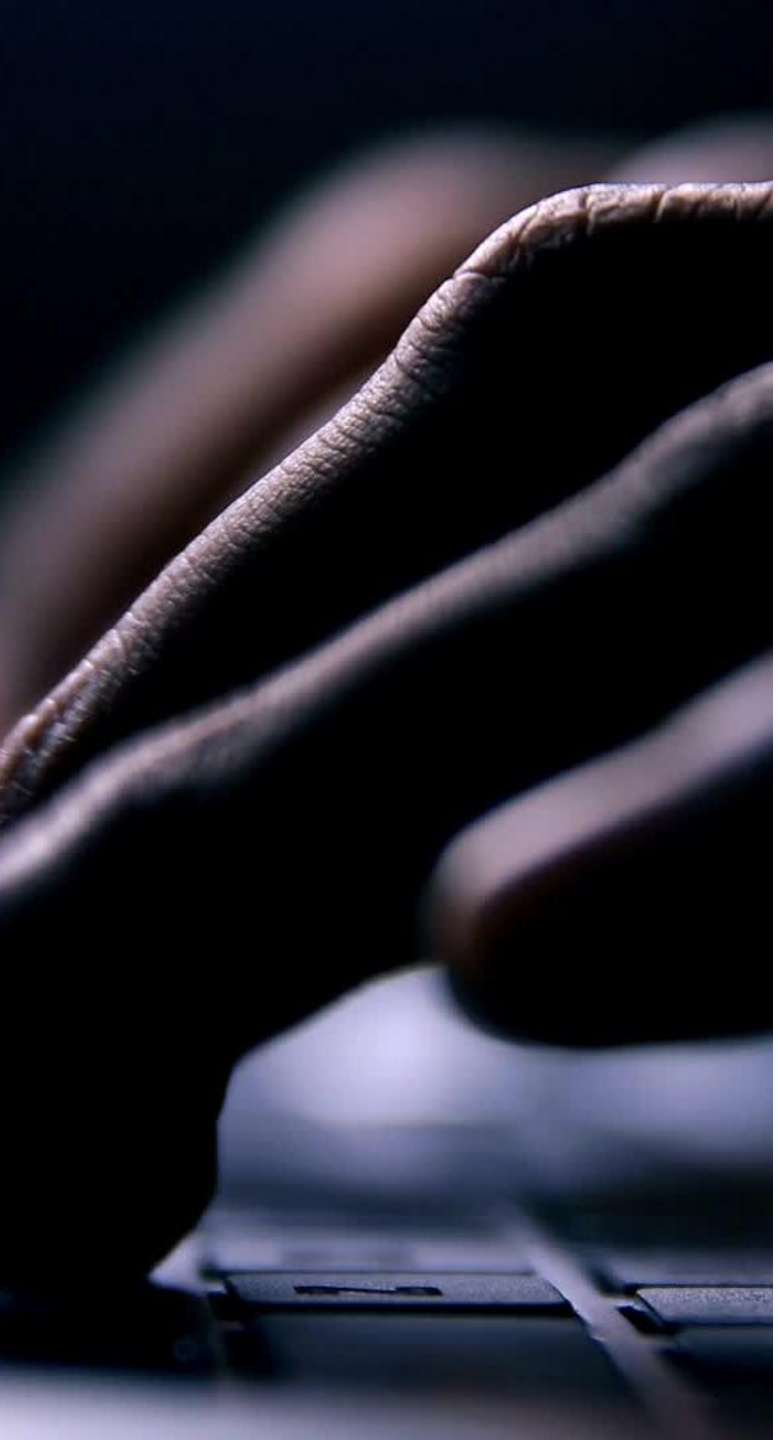


Fatec
Diadema
Luigi Papaiz

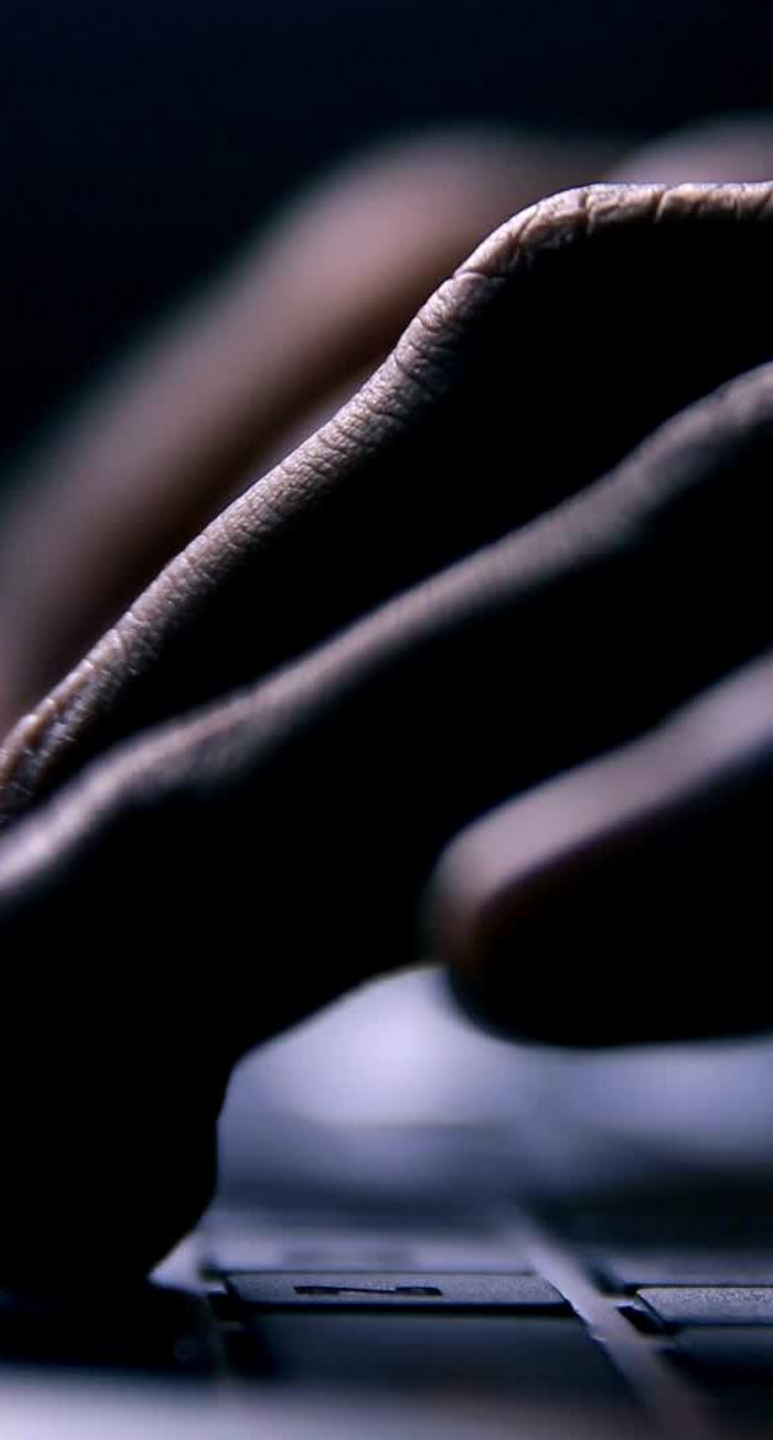
Integração e Entrega Contínua

Prof. Rafael Miranda

Na aula anterior...

Fatec
Diadema
Luigi Papaiz

- Primeira pipeline no GitHub Actions
  - **O que fez a pipeline executar?**
    - **R:** push no repositório
      - commit > push > pipeline executa



Fatec
Diadema
Luigi Papaiz

Git essencial para Ci/CD

O que é Controle de Versão

Fatec
Diadema
Luigi Papaiz

- Controle de versão é um sistema que permite:
  - registrar alterações no código
  - manter histórico de mudanças
  - permitir colaboração entre desenvolvedores
  - recuperar versões anteriores
- Problema sem controle de versão:
  - arquivos sobrescritos
  - perda de código
  - dificuldade de colaboração

Origem do Git

Fatec
Diadema
Luigi Papaiz

- O **Git** foi criado em **2005** por **Linus Torvalds**, o mesmo criador do Linux.
- Motivação:
  - gerenciar o desenvolvimento do **kernel Linux**
  - milhares de desenvolvedores trabalhando ao mesmo tempo
  - necessidade de um sistema **rápido e distribuído**

Repositório

Fatec
Diadema
Luigi Papaiz

- Repositório é o local onde o projeto e seu histórico são armazenados
- Tipos:
  - **Repositório local**
    - computador do desenvolvedor
  - **Repositório remoto**
    - servidor (GitHub, GitLab, Bitbucket)

Inicializando um repositório

Fatec
Diadema
Luigi Papaiz

- O comando **git init** cria um **novo repositório Git local** dentro de uma pasta do projeto.
- Ele cria a estrutura interna do Git (pasta .git) que passa a controlar o versionamento dos arquivos.
  - Resultado:
    - O diretório passa a ser um **repositório Git**
    - O Git cria a pasta oculta .git
    - Uma **branch inicial** é criada (master ou main)

Mudança da Branch inicial

Fatec
Diadema
Luigi Papaiz

- Mudança de **master** para **main** no Git
- **Até 2020**
  - A branch principal do repositório era chamada **master**
  - Era o padrão histórico do Git
- **A partir de 2020**
  - Plataformas como o GitHub passaram a usar **main** como padrão
  - Objetivo: evitar a terminologia **master/slave**
  - **Comando para ajuste**

```
git branch -M main
```

Clonando um repositório

Fatec
Diadema
Luigi Papaiz

- Para trabalhar em um projeto precisamos **clonar o repositório**.
- Comando:

```
git clone URL_DO_REPOSITORIO
```

- Exemplo:

```
git clone https://github.com/exemplo/projeto.git
```

- Isso cria uma cópia local do projeto.

Verificando o estado do repositório

Fatec
Diadema
Luigi Papaiz

- Git permite verificar alterações no projeto.
- Comando:

```
git status
```

- Mostra:
  - arquivos modificados
  - arquivos adicionados
  - arquivos prontos para commit

Branch

Fatec
Diadema
Luigi Papaiz

- Branch é uma linha de desenvolvimento independente.
- Permite desenvolver funcionalidades separadamente.
- Exemplo:
  - main
    - ├─ feature-login
    - └─ feature-api
- Depois as alterações podem ser integradas.

Criando uma branch

Fatec
Diadema
Luigi Papaiz

- Estrutura:

```
main
├ feature-login
└ feature-api
```

- Criar Branch:

```
git checkout -b nome-da-branch
```

- Exemplo:

```
git checkout -b dev
```

Adicionando alterações

Fatec
Diadema
Luigi Papaiz

- Antes do commit precisamos adicionar arquivos ao **staging**.
- Comando:

```
git add .
```

- Ou adicionar arquivo específico:

```
git add arquivo.txt
```

Commit

Fatec
Diadema
Luigi Papaiz

- Commit é o registro de uma alteração no projeto.
  - Cada commit possui:
    - identificador único
    - autor
    - data
    - mensagem descritiva

Criando um commit

Fatec
Diadema
Luigi Papaiz

- Comando:

```
git commit -m "mensagem do commit"
```

- Exemplo:

```
git commit -m "Meu primeiro commit"
```

- Boa prática:
  - **mensagens claras e objetivas.**

Histórico de commits

Fatec
Diadema
Luigi Papaiz

- Git permite visualizar o histórico de alterações do projeto.
- Comando:

```
git log
```

- Exibe informações como:
  - identificador do commit
  - Autor
  - Data
  - mensagem do commit

Git Revert — Desfazendo um commit

Fatec
Diadema
Luigi Papaiz

- git revert desfaz as alterações de um commit criando um novo commit que cancela o anterior, sem alterar o histórico do projeto.
- Comando:
  - Primeiro identifique o commit:

```
git log --oneline
```

  - Depois execute:

```
git revert HASH_DO_COMMIT
```

Merge

Fatec
Diadema
Luigi Papaiz

- Merge é o processo de combinar duas branches.

feature-login

↓

merge

↓

main

- Isso integra as alterações ao projeto principal.

Comando para realizar merge

Fatec
Diadema
Luigi Papaiz

- Primeiro voltar para a branch principal:

```
git checkout main
```

- Depois executar o merge:

```
git merge nome-da-branch
```

- Exemplo:

```
git merge dev
```

Enviando alterações para o GitHub

Fatec
Diadema
Luigi Papaiz

- Para atualizar o repositório remoto usamos:

```
git push
```

- Ou especificando branch:

```
git push origin dev
```

- Nesse momento a pipeline pode ser executada
- Acrescentando –u, não será necessário especificar a branch nas próximas:

```
git push -u origin main
```

Atualizando o repositório local

Fatec
Diadema
Luigi Papaiz

- Se outras pessoas fizeram alterações precisamos atualizar o projeto.

```
git pull
```

- Isso baixa alterações do repositório remoto
- Ou indicar a Branch que quer trazer as alterações:

```
git pull origin main
```

Merge usando GitHub (Pull Request)

Fatec
Diadema
Luigi Papaiz

- Em projetos colaborativos, o merge normalmente não é feito localmente.
- Ele ocorre através de um Pull Request no Git Hub

base: main
compare: dev

O que é um Pull Request

Fatec
Diadema
Luigi Papaiz

- Pull Request não é um comando Git.
- Ele é um **mecanismo de colaboração do GitHub**.
- Significa:
  - "Eu quero que você **puxe (pull)** minhas mudanças para a branch principal".
- Ou seja, a pessoa está pedindo:

Meu código → seja integrado ao projeto

Fluxo de trabalho com Git

Fatec
Diadema
Luigi Papaiz

- Fluxo comum de desenvolvimento:

clone
↓
criar branch
↓
editar código
↓
add
↓
commit
↓
push
↓
pull request
↓
merge

Git e CI/CD

Fatec
Diadema
Luigi Papaiz

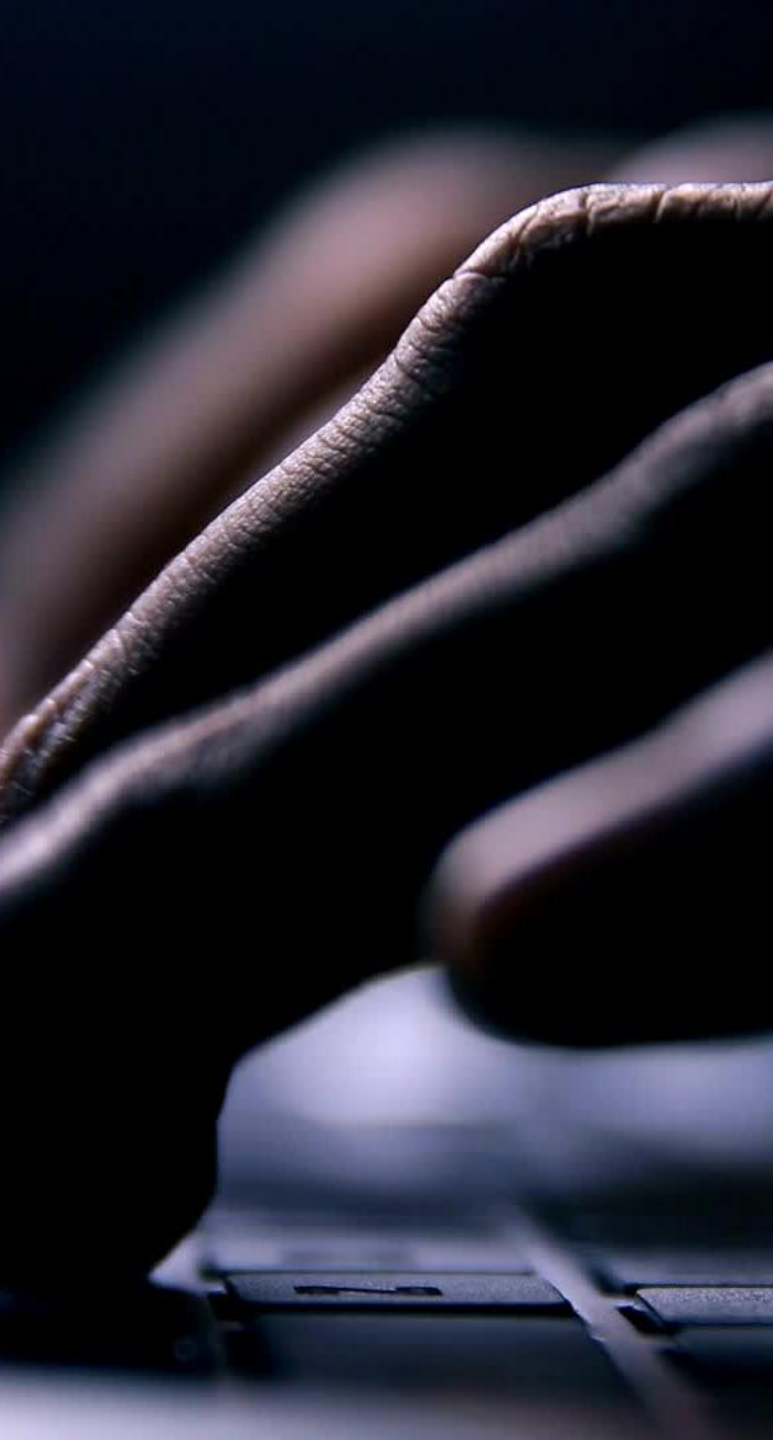
- Pipelines normalmente são executadas após eventos no Git.
  - Exemplo de configuração:

```
on:
 push:
  branches:
    - main
 pull_request:
   branches:
    - main
```

Boas práticas de Git

Fatec
Diadema
Luigi Papaiz

- Boas práticas no uso do Git:
  - commits pequenos
  - mensagens de commit claras
  - uso de branches para funcionalidades
  - uso de Pull Requests
  - evitar commits diretos na main



Fatec
Diadema
Luigi Papaiz

Evolução da Pipeline

Qual prática adotamos no Pipeline criado na última aula?

Fatec
Diadema
Luigi Papaiz

- Nosso pipeline:
  - Build automático
  - Testes automáticos
  - Deploy automático
  - Sem aprovação
- **Classificação:**
  - CD - Continuous Deployment (Implantação Contínua)

Transformando em Continuous Delivery (Entrega Contínua)

Fatec
Diadema
Luigi Papaiz

- Introduzindo Environment (Ambiente)
- Criar no GitHub:
  - **Settings → Environments → production**
- Environment permite:
  - Controle de deploy
  - Regras de proteção
  - Aprovação manual

Acrescentar no YAML

```
deploy:
  runs-on: ubuntu-latest
  needs: test
  environment:
    name: production
```

Required Reviewers

Fatec
Diadema
Luigi Papaiz

- No environment configurar:
  - Required reviewers
- Sem aprovação → deploy não executa.
- Agora temos um “gate”.

Novo Fluxo com Aprovação

Fatec
Diadema
Luigi Papaiz

- Push
  - Build
  - Testes
  - Pipeline pausa
  - Aguarda aprovação
  - Deploy executa
- Isso caracteriza Continuous Delivery (Entrega Contínua).

Comparação Visual

Fatec
Diadema
Luigi Papaiz

- **Continuous Deployment**

  Push → Build → Test → Deploy automático

- **Continuous Delivery**

  Push → Build → Test → Aguarda aprovação → Deploy

Aplicação no Mundo Real

Fatec
Diadema
Luigi Papaiz

- Empresas utilizam aprovação para:
  - Gestão de risco
  - Compliance
  - Auditoria
  - Governança
  - Controle de mudanças
- CI/CD não é apenas técnico. É também organizacional.

O que é Build?

Fatec
Diadema
Luigi Papaiz

- Build é a transformação do código-fonte em algo executável.
- Pode incluir:
  - Compilação
  - Empacotamento
  - Preparação do ambiente
  - Geração de artefato

Evolução do Pipeline

Fatec
Diadema
Luigi Papaiz

- Stages Build, Test e Deploy separados
- Build gerar artefato
  - Artefato é o resultado do processo de build.
    - Exemplos:
      - .jar (Java)
      - .exe (C#)
      - .whl (Python)
      - .zip (pacote distribuível)
      - Docker Image

Novo Fluxo do Pipeline

Fatec
Diadema
Luigi Papaiz

- Código:
  - Build
    - Gera Artefato
    - Publica Artefato
  - Test
    - Valida o artefato
  - Deploy
    - Entrega o artefato

Como fica o YAML

Fatec
Diadema
Luigi Papaiz

Acrescentar steps no Build

```
steps:
  - name: Gerar artefato ZIP
    run: zip -r pipeline-demo.zip .

  - name: Publicar artefato
    uses: actions/upload-artifact@v3
      with:
        name: codigo-buildado
        path: pipeline-demo.zip
```

Acrescentar steps no test

```
steps:
  - name: Baixar artefato
    uses: actions/download-artifact@v3
      with:
        name: codigo-buildado

  - name: Extrair artefato
    run: unzip pipeline-demo.zip
```

Feedback no Pipeline

Fatec
Diadema
Luigi Papaiz

- Pipeline é um mecanismo de retorno automático e imediato sobre o estado do sistema.
- Ele fornece:
  - Status visual (verde/vermelho)
  - Logs detalhados
  - Bloqueio de merge
  - Notificações integradas

Feedback por Stage

Fatec
Diadema
Luigi Papaiz

- Cada stage responde uma pergunta:
  - Build → Conseguimos gerar artefato?
  - Test → O comportamento está correto?
  - Deploy → Podemos entregar com segurança?

Por que Feedback é Essencial?

Fatec
Diadema
Luigi Papaiz

- Feedback rápido reduz custo do erro.
- Erro detectado:
  - No commit → barato
  - Em produção → caro
- CI existe para reduzir o tempo entre:
- Erro → Descoberta → Correção